

Лекция 1

«Введение в тестирование»

Задание:

- «1.1. Обеспечение качества и тестирование программного обеспечения»: переписать все определения;
- Прочитать полностью лекцию;
- Ответить на вопросы письменно.

1.1. Обеспечение качества и тестирование программного обеспечения - основные понятия и определения

Качество программного обеспечения (Software Quality) — это степень, в которой программное обеспечение обладает требуемой комбинацией свойств.

Качество программного обеспечения (Software Quality) — это совокупность характеристик программного обеспечения, относящихся к его способности удовлетворять установленные и предполагаемые потребности.

Обеспечение качества (Quality Assurance - QA) — это совокупность мероприятий, охватывающих все технологические этапы разработки, выпуска и эксплуатации программного обеспечения (ПО) информационных систем, предпринимаемых на разных стадиях жизненного цикла ПО для обеспечения требуемого уровня качества выпускаемого продукта.

Контроль качества (Quality Control - QC) — это совокупность действий, проводимых над продуктом в процессе разработки для получения информации о его актуальном состоянии в разрезе: "готовность продукта к выпуску", "соответствие зафиксированным требованиям", "соответствие заявленному уровню качества продукта".

Тестирование программного обеспечения (Software Testing) — это одна из техник контроля качества, включающая в себя активности по планированию работ (Test Management), проектированию тестов (Test Design), выполнению тестирования (Test Execution) и анализу полученных результатов (Test Analysis).

Верификация (verification) — это процесс оценки системы или её компонентов с целью определения, удовлетворяют ли результаты текущего этапа разработки условиям, сформированным в начале этого этапа. Т.е. выполняются ли наши цели, сроки, задачи по разработке проекта, которые были определены в начале текущей фазы.

Валидация (validation) — это определение соответствия разрабатываемого ПО ожиданиям и потребностям пользователя, требованиям к системе.

План Тестирования (Test Plan) — это документ, который описывает весь объем работ по тестированию, начиная с описания объекта, стратегии, расписания, критериев начала и окончания тестирования до необходимого в процессе работы

оборудования, специальных знаний, а также оценки рисков с вариантами их разрешения.

Тест дизайн (Test Design) — это этап процесса тестирования ПО, на котором проектируются и создаются тестовые случаи (тест-кейсы), в соответствии с определенными ранее критериями качества и целями тестирования.

Тестовый случай (Test Case) — это артефакт, описывающий совокупность шагов, конкретных условий и параметров, необходимых для проверки реализации тестируемой функции или её части.

Баг/Дефект Репорт (Bug Report) — это документ, описывающий ситуацию или последовательность действий, приведшую к некорректной работе объекта тестирования с указанием причин и ожидаемого результата.

Тестовое Покрытие (Test Coverage) — это одна из метрик оценки качества тестирования, представляющая из себя плотность покрытия тестами требований либо исполняемого кода.

Детализация Тест-Кейсов (Test Case Specification) — это уровень детализации описания тестовых шагов и требуемого результата, при котором обеспечивается разумное соотношение времени прохождения к тестовому покрытию.

Время Прохождения Тест Кейса (Test Case Pass Time) — это время от начала прохождения шагов тест-кейса до получения результата теста.

1.2. Качество программного обеспечения

Каждый день в своей работе мы сталкиваемся с достаточно абстрактным понятием «качество ПО» и, если задать вопрос тестировщику или программисту «что такое качество?», то у каждого найдется своё толкование. Рассмотрим определение "качества ПО" в контексте международных стандартов:

Качество программного обеспечения — это степень, в которой ПО обладает требуемой комбинацией свойств.

Качество программного обеспечения — это совокупность характеристик ПО, относящихся к его способности удовлетворять установленные и предполагаемые потребности.

Характеристики качества ПО

Функциональность (Functionality) - определяется способностью ПО решать задачи, которые соответствуют зафиксированным и предполагаемым потребностям пользователя, при заданных условиях использования ПО. Т.е. эта характеристика отвечает то, что ПО работает исправно и точно, функционально совместимо соответствует стандартам отрасли и защищено от несанкционированного доступа.

Надежность (Reliability) – способность ПО выполнять требуемые задачи в обозначенных условиях на протяжении заданного промежутка времени или указанное количество операций. Атрибуты данной характеристики – это завершенность и целостность всей системы, способность самостоятельно и корректно восстанавливаться после сбоев в работе, отказоустойчивость.

Удобство использования (Usability) – возможность легкого понимания, изучения, использования и привлекательности ПО для пользователя.

Эффективность (Efficiency) – способность ПО обеспечивать требуемый уровень производительности, в соответствии с выделенными ресурсами, временем и другими обозначенными условиями.

Удобство сопровождения (Maintainability) – легкость, с которой ПО может анализироваться, тестироваться, изменяться для исправления дефектов для реализации новых требований, для облегчения дальнейшего обслуживания и адаптирования к имеющемуся окружению.

Портативность (Portability) – характеризует ПО с точки зрения легкости его переноса из одного окружения (software/ hardware) в другое.

Модель качества программного обеспечения

На данный момент, наиболее распространена и используется многоуровневая модель качества программного обеспечения, представленная в наборе стандартов ISO 9126. На верхнем уровне выделено 6 основных характеристик качества ПО, каждую из которых определяют набором атрибутов, имеющих соответствующие метрики для последующей оценки.



Рисунок 1 - Модель качества программного обеспечения (ISO 9126-1)

1.3. Кто такой тестировщик и что он делает

В начале карьеры любой специалист (и тестировщик не является исключением) является исполнителем и учеником. Достаточно хорошо понимать, что такое тест-кейсы, отчёты о дефектах, уметь читать требования, пользоваться парой инструментальных средств и хорошо уживаться в команде.

Постепенно тестировщик начинает погружаться во все стадии разработки проекта, понимая их всё полнее и полнее, начинает не только активно использовать, но и разрабатывать проектную документацию, принимать всё более ответственные решения.

Если выразить образно главную цель тестировщика, то она будет звучать так: «понимать, что в настоящий момент необходимо проекту, получает ли проект это необходимое в должной мере и, если нет, то как изменить ситуацию к лучшему». Звучит похоже на цель руководителя проекта, верно? Верно. Начиная с некоторого уровня развития, IT-специалисты, по большому счёту, различаются лишь наборами технических навыков и основной областью приложения этих навыков.

1. Знание иностранных языков. Да, это не технический навык. Но, тем не менее, он идёт под номером «ноль». Можете считать это аксиомой: «нет знания английского — нет карьеры в IT». Другие иностранные языки тоже приветствуются, но английский первичен.

2. Уверенное владение компьютером на уровне по-настоящему продвинутого пользователя и желание постоянно развиваться в этой области. Можете ли Вы представить себе профессионального повара, который не может пожарить картошку (не «не обязан», а «не умеет в принципе»)? Выглядит странно? Не менее странно выглядит «IT'шник» (именно так, в кавычках), неспособный набрать вменяемо отформатированный текст, скопировать файл по сети, развернуть виртуальную машину или выполнить любое иное повседневное рутинное действие.

3. Программирование. Оно на порядок упрощает жизнь любому IT'шнику, а тестировщику — в первую очередь. Можно ли тестировать без знания программирования? Да, можно. Можно ли это делать по-настоящему хорошо? Нет. И сейчас самый главный (почти религиозно-философский) вопрос: какой язык программирования изучать? C/C++/C#, Java, PHP, JavaScript, Python, Ruby и т.д. — начинайте с того, на чём написан ваш проект.

4. Базы данных и язык SQL. Здесь от тестировщика тоже не требуется квалификация на уровне узких специалистов, но минимальные навыки работы с наиболее распространёнными СУБД и умение писать простые запросы можно считать обязательными.

5. Понимание принципов работы сетей и операционных систем. Хотя бы на минимальном уровне, позволяющем провести диагностику проблемы и решить её своими силами, если это возможно.

6. Понимание принципов работы веб-приложений и мобильных приложений. В наши дни почти всё пишется именно в виде таких приложений, и понимание соответствующих технологий становится обязательным для эффективного тестирования.

Откуда берутся ошибки в ПО?

Почему бывает так, что программы работают неправильно? Все очень просто – они создаются и используются людьми. Если пользователь допустит ошибку, то это может привести к проблеме в работе программы – она используется неправильно, значит, может повести себя не так, как ожидалось.

Ошибка (error) – это действие человека, которое порождает неправильный результат.

Однако программы разрабатываются и создаются людьми, которые также могут допускать (и допускают) ошибки. Это значит, что недостатки есть и в самом программном обеспечении. Они называются дефектами или багами (оба обозначения равносильны). Здесь важно помнить, что программное обеспечение – нечто большее, чем просто код.

Дефект, Баг (Defect, Bug) – недостаток компонента или системы, который может привести к отказу определенной функциональности. Дефект, обнаруженный во время исполнения программы, может вызвать отказ отдельного компонента или всей системы.

При исполнении кода программы дефекты, заложенные еще во время его написания, могут проявиться: программа может не делать того, что должна или наоборот делать то, чего не должна – происходит сбой.

Сбой (failure) – несоответствие фактического результата (actual result) работы компонента или системы ожидаемому результату (expected result).

Сбой в работе программы может являться индикатором наличия в ней дефекта.

Таким образом, баг существует при одновременном выполнении трех условий:

- известен ожидаемый результат;
- известен фактический результат;
- фактический результат отличается от ожидаемого результата.

Важно понимать, что не все баги становятся причиной сбоев – некоторые из них могут никак себя не проявлять и оставаться незамеченными (или проявляться только при очень специфических обстоятельствах).

Причиной сбоев могут быть не только дефекты, но также и условия окружающей среды: например, радиация, электромагнитные поля или загрязнение также могут влиять на работу как программного, так и аппаратного обеспечения.

Всего существует несколько источников дефектов и, соответственно, сбоев:

- ошибки в спецификации, дизайне или реализации программной системы;
- ошибки использования системы;
- неблагоприятные условия окружающей среды;
- умышленное причинение вреда;
- потенциальные последствия предыдущих ошибок, условий или умышленных действий.

Дефекты могут возникать на разных уровнях, и от того, будут ли они исправлены и когда, будет напрямую зависеть качество системы.

Качество (Quality) – степень, в которой совокупность присущих характеристик соответствует требованиям.

Качество программного обеспечения (Software Quality) – это совокупность характеристик программного обеспечения, отражающих его способность удовлетворять установленные и предполагаемые потребности.

Требование (Requirement) – потребность или ожидание, которое установлено. Обычно предполагается или является обязательным.

В первом случае все было сделано правильно, и мы получили продукт, полностью соответствующий ожиданиям заказчика и удовлетворяющий критериям качества.

Во втором случае ошибки были допущены уже при кодировании, что привело к появлению дефектов в готовом продукте. Но на этом уровне баги достаточно легко обнаружить и исправить, поскольку мы видим несоответствие требованиям.

Третий вариант хуже – здесь ошибки были допущены на этапе проектирования системы. Заметить это можно лишь проведя тщательную сверку со спецификацией. Исправить такие дефекты тоже не просто – нужно заново перерабатывать дизайн продукта.

В четвертом случае дефекты были заложены еще на этапе формирования требований; вся дальнейшая разработка и даже тестирование пошли по изначально неправильному пути. Во время тестирования мы не найдем багов – программа пройдет все тесты, но может быть забракована заказчиком.

Условно можно выделить **пять причин появления дефектов в программном коде**.

1. Недостаток или отсутствие общения в команде. Зачастую бизнес-требования просто не доходят до команды разработки. У заказчика есть понимание того, каким он хочет видеть готовый продукт, но, если должным образом не объяснить его идею разработчикам и тестировщикам, результат может оказаться не таким, как предполагалось. Требования должны быть доступны и понятны всем участникам процесса разработки ПО.

2. Сложность программного обеспечения. Современное ПО состоит из множества компонентов, которые объединяются в сложные программные системы. Многопоточные приложения, клиент-серверная и распределенная архитектура, многоуровневые базы данных – программы становятся все сложнее в написании и поддержке, и тем труднее становится работа программистов. А чем труднее работа, тем больше ошибок может допустить исполняющий ее человек.

3. Изменения требований. Даже незначительные изменения требований на поздних этапах разработки требуют большого объема работ по внесению изменений в систему. Меняется дизайн и архитектура приложения, что, в свою очередь, требует внесения изменений в исходный код и принципы взаимодействия программных модулей. Такие текущие изменения зачастую становятся источником трудноуловимых дефектов. Тем не менее, часто меняющиеся требования в

современном бизнесе – скорее правило, чем исключение, поэтому непрерывное тестирование и контроль рисков в таких условиях – прямая обязанность специалистов отдела обеспечения качества.

4. Плохо документированный код. Сложно поддерживать и изменять плохо написанный и слабо документированный программный код. Во многих компаниях существуют специальные правила по написанию и документированию кода программистами. Хотя на практике часто бывает так, что разработчики вынуждены писать программы в первую очередь быстро, а это сказывается на качестве продукта.

5. Средства разработки ПО. Средства визуализации, библиотеки, компиляторы, генераторы скриптов и другие вспомогательные инструменты разработки – это тоже зачастую плохо работающие и слабо документированные программы, которые могут стать источником дефектов в готовом продукте.

Вопросы

1. Что такое качество программного обеспечения (Software Quality) и как оно определяется в контексте международных стандартов?
2. Какие основные характеристики качества программного обеспечения выделяют в стандарте ISO 9126?
3. В чем разница между обеспечением качества (Quality Assurance - QA) и контролем качества (Quality Control - QC)?
4. Какие основные этапы включает в себя тестирование программного обеспечения (Software Testing)?
5. Что такое верификация (verification) и валидация (validation) в контексте разработки программного обеспечения?
6. Какие основные компоненты включает в себя план тестирования (Test Plan)?
7. Что такое тест-кейс (Test Case) и какие элементы он включает?
8. Какие источники дефектов могут приводить к сбоям в работе программного обеспечения?
9. Какие основные причины появления дефектов в программном коде выделяются в лекции?
10. Какие технические навыки и знания считаются обязательными для эффективного тестирования программного обеспечения?